AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247
Batch: 37 **Assignment-** 11.2

Task 1: Stack Using AI Guidance

Prompt Used:

"#Generate a Python Stack class supporting push, pop, peek, and is_empty operations with proper documentation."

```
        Returns:
        The item at the top of the stack.

        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
        """
        Return the item at the top of the stack without removing it.

        Returns:
        The item at the top of the stack.

        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
```

```
AI Assist 11.2.py > Stack > pop
class Stack:
    def is_empty(self):
        """
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> & C:/Users/pulla/AppData/Local/Programs
c:/Users/pulla/OneDrive/Documents/SRU/AI Asisted Coding/AI Assist 11.2.py"
1
True
```

Implementation Summary:

AI generated a Stack class using a list to store elements.

Methods implemented include push (add element), pop (remove top), peek (view top), and is_empty (check stack status).

Output:

Stack operations worked correctly following LIFO principle.

Observation:

AI provided clear structure and documentation.

The implementation is simple, efficient, and easy to understand.

Task 2: Queue Design

Prompt Used:

"#Create a Python Queue data structure following FIFO principles with enqueue, dequeue, front, and size methods."

```python
#Create a Python Queue data structure following FIFO principles with enqueue, dequeue, front, and size methods.

class Queue:
    """
    A Queue is a data structure that follows the First In First Out (FIFO) principle.
    This Queue class supports enqueue, dequeue, front, and size operations.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """
        Add an item to the end of the queue.

        Parameters:
        item: The item to be added to the queue.
        """
        self.items.append(item)

    def dequeue(self):
```

```python
class Queue:
    def dequeue(self):
        Remove and return the item at the front of the queue.

        Returns:
        The item at the front of the queue.

        Raises:
        IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)

    def front(self):
        """
        Return the item at the front of the queue without removing it.

        Returns:
        The item at the front of the queue.

        Raises:
        IndexError: If the queue is empty.
```

```python
class Queue:
    def front(self):
        if self.is_empty():
            raise IndexError("Front from an empty queue")
        return self.items[0]

    def size(self):
        """
        Return the number of items in the queue.

        Returns:
        The number of items in the queue.
        """
        return len(self.items)

    def is_empty(self):
        """
        Check if the queue is empty.

        Returns:
        True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0
```

AI Assist 11.2.py > ...

```python
class Queue:
    def is_empty(self):
        """
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print(queue.front())  # Output: 1
    print(queue.size())   # Output: 3
    print(queue.dequeue())   # Output: 1
    print(queue.is_empty())  # Output: False
    print(queue.dequeue())   # Output: 2
    print(queue.dequeue())   # Output: 3
    print(queue.is_empty())  # Output: True
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
2
3
True
```

Implementation Summary:

AI implemented the Queue using a list to maintain insertion order.

Methods include enqueue (add element), dequeue (remove first element), front (view first element), and size calculation.

Output:

Queue operations followed FIFO behavior correctly.

Observation:

AI-generated implementation was logically correct and readable.
Proper method naming improved clarity.

Task 3: Singly Linked List Construction

Prompt Used:

"#Build a singly linked list in Python supporting node creation, insertion, and traversal."

```python
# Build a singly linked list in Python supporting node creation, insertion, and traversal.
class Node:
    """
    A Node in a singly linked list.
    """

    def __init__(self, data):
        """Initialize a node with data and a pointer to the next node."""
        self.data = data
        self.next = None
class SinglyLinkedList:
    """
    A Singly Linked List is a data structure that consists of nodes where each node points to the next node in th
    This SinglyLinkedList class supports node creation, insertion, and traversal.
    """

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """
        Insert a new node with the given data at the end of the linked list.
```

AI Assist 11.2.py > ...

```python
class SinglyLinkedList:
    def traverse(self):
        current_node = self.head
        while current_node:
            elements.append(current_node.data)
            current_node = current_node.next
        return elements
# Example usage:
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)

    print(linked_list.traverse())  # Output: [1, 2, 3]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
...
True
[1, 2, 3]
```

```python
class SinglyLinkedList:
    def insert(self, data):
        Parameters:
        data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def traverse(self):
        """
        Traverse the linked list and return a list of node data.

        Returns:
        A list containing the data of each node in the linked list.
        """
        elements = []
        current_node = self.head
        while current_node:
```

Implementation Summary:

AI created a Node class and LinkedList class.

Insertion logic connects new nodes sequentially, and traversal displays elements.

Output:

Linked list successfully inserted nodes and displayed elements in order.

Observation:

AI correctly handled pointer references and node linking.

The structure demonstrates fundamental linked list operations clearly.

Task 4: Binary Search Tree Operations

Prompt Used:

“#Implement a Binary Search Tree with insertion and in-order traversal.”

```python
#Implement a Binary Search Tree with insertion and in-order traversal.
class TreeNode:
    """
    A TreeNode in a binary search tree.
    """

    def __init__(self, data):
        """Initialize a tree node with data and pointers to left and right children."""
        self.data = data
        self.left = None
        self.right = None
class BinarySearchTree:
    """
    A Binary Search Tree is a data structure that consists of nodes where each node has at most two children.
    The left child contains values less than the parent node, and the right child contains values greater than
    This BinarySearchTree class supports insertion and in-order traversal.
    """

    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, data):
        """
```

```python
class BinarySearchTree:
    def insert(self, data):
        """
        Insert a new node with the given data into the binary search tree.

        Parameters:
        data: The data to be stored in the new node.
        """
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        """Helper method to insert a new node recursively."""
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
            else:
```

```python
            else:
                self._insert_recursive(node.right, data)

    def in_order_traversal(self):
        """
        Perform in-order traversal of the binary search tree.

        Returns:
        A list containing the data of each node in in-order sequence.
        """
        elements = []
        self._in_order_recursive(self.root, elements)
        return elements

    def _in_order_recursive(self, node, elements):
        """Helper method to perform in-order traversal recursively."""
        if node:
            self._in_order_recursive(node.left, elements)
            elements.append(node.data)
            self._in_order_recursive(node.right, elements)
# Example usage:
```

AI Assist 11.2.py > ...

```python
# Example usage:
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)

    print(bst.in_order_traversal())  # Output: [2, 3, 4, 5, 7]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> & C:/Users/pulla/AppData/Local/Pr
c:/Users/pulla/OneDrive/Documents/SRU/AI Asisted Coding/AI Assist 11.2.py"
[2, 3, 4, 5, 7]
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding>
```

Implementation Summary:

AI generated a BST class with recursive insertion logic.
In-order traversal was implemented to display sorted output.

Output:

Nodes were inserted correctly and displayed in sorted order using in-order traversal.

Observation:

AI-assisted implementation maintained BST properties.
Recursive logic was clear and functionally correct.

Task 5: Hash Table Implementation

Prompt Used:

"#Create a hash table in Python with collision handling using

chaining.”

```python
#Create a hash table in Python with collision handling using chaining.
class HashTable:
    """
    A Hash Table is a data structure that maps keys to values for efficient lookup.
    This HashTable class supports collision handling using chaining.
    """

    def __init__(self, size=10):
        """Initialize a hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.

        Parameters:
        key: The key to be inserted.
        value: The value associated with the key.
```

```python
def insert(self, key, value):
    index = self._hash(key)
    # Check if the key already exists and update the value
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            self.table[index][i] = (key, value)
            return
    # If the key does not exist, add a new key-value pair
    self.table[index].append((key, value))

def get(self, key):
    """
    Retrieve the value associated with the given key.

    Parameters:
    key: The key to look up.

    Returns:
    The value associated with the key.

    Raises:
    KeyError: If the key is not found in the hash table.
    """
```

```python
class HashTable:
    def get(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        raise KeyError("Key not found")

    def remove(self, key):
        """
        Remove a key-value pair from the hash table.

        Parameters:
        key: The key to be removed.

        Raises:
        KeyError: If the key is not found in the hash table.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return
        raise KeyError("Key not found")
```

AI Assist 11.2.py > HashTable > insert

```python
# Example usage:
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print(hash_table.get("name"))  # Output: Alice
    print(hash_table.get("age"))   # Output: 30

    hash_table.remove("city")
    try:
        print(hash_table.get("city"))  # This will raise a KeyError
    except KeyError as e:
        print(e)  # Output: Key not found
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> & C:/Users/pulla/AppData/L
c:/Users/pulla/OneDrive/Documents/SRU/AI Asisted Coding/AI Assist 11.2.py"
[1, 2, 3]
[2, 3, 4, 5, 7]
Alice
30
'Key not found'
```

Implementation Summary:

AI implemented the hash table using an array of lists (chaining method).
Insert, search, and delete operations were supported.

Output:

Hash table correctly handled collisions and retrieved stored values.

Observation:

AI suggested efficient collision handling technique.
Chaining improved reliability of hash operations.

Conclusion: This lab demonstrates how AI-assisted coding simplifies the implementation of fundamental data structures while maintaining correctness and clarity.